DEAKIN UNIVERSITY

SIT223-SIT753
Pass Task:
Building a Jenkins Continuous Delivery Pipeline

Building a Jenkins Continuous Delivery Pipeline

This is an individual Pass task.

Overview

The objective of this assignment is to design and implement a generic Jenkins pipeline that can be used to automate the continuous delivery of software applications.

Please note: For this pass-level task, you are only required to print out a message at each stage of the pipeline (e.g., build, test, etc.). Integration of tools is not necessary.

Task Instructions

Part 1

1. Create a new Jenkins pipeline job.

2. Define the pipeline script in the Jenkinsfile.

3. The pipeline must have six stages with the following names:
    a. Build
    b. Test
    c. Code Quality Check
    d. Deploy
    e. Approval
    f. Deploy to Production

4. Define and use environment variables for the pipeline. The following environment variables must be included:
    a. DIRECTORY_PATH: The path of the code directory being fetched.
    b. TESTING_ENVIRONMENT: The name of the testing environment.
    c. PRODUCTION_ENVIRONMENT: The name of the production environment. Use your name as the name of production environment.

5. Configure the pipeline stages.
    a. In the **Build** stage, configure the pipeline to print a message that reads: *"Fetch the source code from the directory path specified by the environment variable"*. Add another build step to the job to print this message: *"Compile the code and generate any necessary artefacts"*.
    b. In the **Test** stage, add steps to print the messages *"Unit tests"* and *"Integration tests"*.

c. In the **Code** Quality Check stage, add a message showing that *"Check the quality of the code".*
d. In the **Deploy** stage, print *"Deploy the application to a testing environment specified by the environment variable"*.
e. In the **Approval** stage, add a sleep command to simulate manual approval. The pipeline must pause for 10 seconds before continuing.
f. In the **Deploy to Production** stage, add one step to display a message related to deploying the code to the production environment, using the environment variable specifying the environment name.

6. Run the pipeline and verify that it executes successfully.

Part 2 (One paragraph)

Name three security tools that can be integrated into a DevOps pipeline to enhance or assess the security of an application, such as checking for code vulnerabilities, conducting application security testing, and ensuring compliance and monitoring. Then, briefly discuss how these tools could be used to address cybersecurity concerns, drawing from your own experience (e.g., professional work or assignments from other units).

Task Submission Instructions:

You must use the provided template found in OnTrack Resources for this task. Submit one PDF file.